

Get To The Point: Summarization with Pointer-Generator Networks

Abigail See
Stanford University
abisee@stanford.edu

Peter J. Liu
Google Brain
peterjliu@google.com

Christopher D. Manning
Stanford University
manning@stanford.edu

Abstract

Neural sequence-to-sequence models have provided a viable new approach for *abstractive* text summarization (meaning they are not restricted to simply selecting and rearranging passages from the original text). However, these models have two shortcomings: they are liable to reproduce factual details inaccurately, and they tend to repeat themselves. In this work we propose a novel architecture that augments the standard sequence-to-sequence attentional model in two orthogonal ways. First, we use a hybrid pointer-generator network that can copy words from the source text via *pointing*, which aids accurate reproduction of information, while retaining the ability to produce novel words through the *generator*. Second, we use *coverage* to keep track of what has been summarized, which discourages repetition. We apply our model to the *CNN / Daily Mail* summarization task, outperforming the current abstractive state-of-the-art by at least 2 ROUGE points.

1 Introduction

Summarization is the task of condensing a piece of text to a shorter version that contains the main information from the original. There are two broad approaches to summarization: *extractive* and *abstractive*. *Extractive methods* assemble summaries exclusively from passages (usually whole sentences) taken directly from the source text, while *abstractive methods* may generate novel words and phrases not featured in the source text – as a human-written abstract usually does. The extractive approach is easier, because copying large

Original Text (truncated): lagos, nigeria (cnn) a day after winning nigeria's presidency, <i>muhammadu buhari</i> told cnn's christiane amannpour that he plans to aggressively fight corruption that has long plagued nigeria and go after the root of the nation's unrest. <i>buhari</i> said he'll "rapidly give attention" to curbing violence in the northeast part of nigeria, where the terrorist group boko haram operates. by cooperating with neighboring nations chad, cameroon and niger, he said his administration is confident it will be able to thwart criminals and others contributing to nigeria's instability. for the first time in nigeria's history, the opposition defeated the ruling party in democratic elections. <i>buhari</i> defeated incumbent goodluck jonathan by about 2 million votes, according to nigeria's independent national electoral commission. the win comes after a long history of military rule, coups and botched attempts at democracy in africa's most populous nation.
Baseline Seq2Seq + Attention: UNK UNK says his administration is confident it will be able to destabilize nigeria's economy. UNK says his administration is confident it will be able to thwart criminals and other nigerians. he says the country has long nigeria and nigeria's economy.
Pointer-Gen: <i>muhammadu buhari</i> says he plans to aggressively fight corruption in the northeast part of nigeria. he says he'll "rapidly give attention" to curbing violence in the northeast part of nigeria. he says his administration is confident it will be able to thwart criminals.
Pointer-Gen + Coverage: <i>muhammadu buhari</i> says he plans to aggressively fight corruption that has long plagued nigeria. he says his administration is confident it will be able to thwart criminals. the win comes after a long history of military rule, coups and botched attempts at democracy in africa's most populous nation.

Figure 1: Comparison of output of 3 abstractive summarization models on a news article. The baseline model makes **factual errors**, a **nonsensical sentence** and struggles with OOV words *muhammadu buhari*. The pointer-generator model is accurate but **repeats itself**. Coverage eliminates repetition. The final summary is composed from **several fragments**.

chunks of text from the source document ensures baseline levels of grammaticality and accuracy. On the other hand, sophisticated abilities that are crucial to high-quality summarization, such as paraphrasing, generalization, or the incorporation of real-world knowledge, are possible only in an abstractive framework (see Figure 5).

Due to the difficulty of abstractive summarization, the great majority of past work has been extractive (Kupiec et al., 1995; Paice, 1990; Sagion and Poibeau, 2013). However, the recent success of *sequence-to-sequence* models (Sutskever

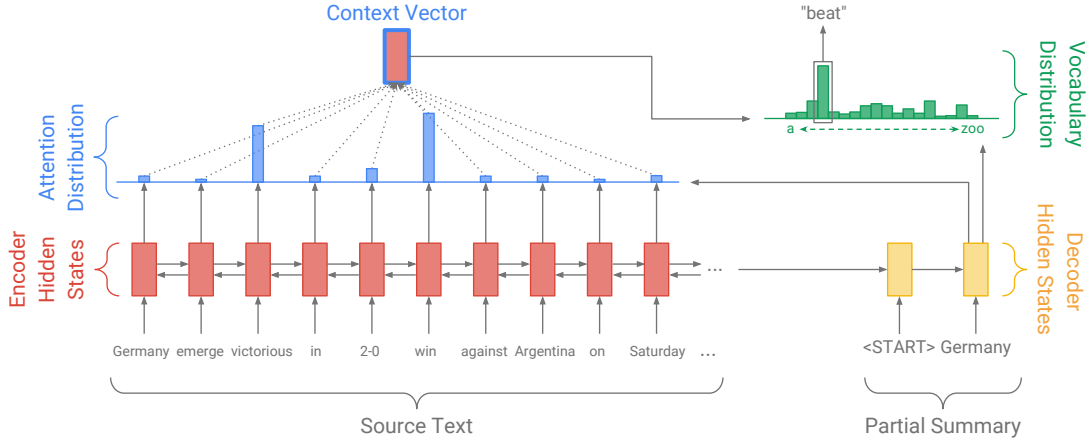


Figure 2: Baseline sequence-to-sequence model with attention. The model may attend to relevant words in the source text to generate novel words, e.g., to produce the novel word *beat* in the abstractive summary *Germany beat Argentina 2-0* the model may attend to the words *victorious* and *win* in the source text.

et al., 2014), in which recurrent neural networks (RNNs) both read and freely generate text, has made abstractive summarization viable (Chopra et al., 2016; Nallapati et al., 2016; Rush et al., 2015; Zeng et al., 2016). Though these systems are promising, they exhibit undesirable behavior such as inaccurately reproducing factual details, an inability to deal with out-of-vocabulary (OOV) words, and repeating themselves (see Figure 1).

In this paper we present an architecture that addresses these three issues in the context of multi-sentence summaries. While most recent abstractive work has focused on headline generation tasks (reducing one or two sentences to a single headline), we believe that longer-text summarization is both more challenging (requiring higher levels of abstraction while avoiding repetition) and ultimately more useful. Therefore we apply our model to the recently-introduced *CNN/Daily Mail* dataset (Hermann et al., 2015; Nallapati et al., 2016), which contains news articles (39 sentences on average) paired with multi-sentence summaries, and show that we outperform the state-of-the-art abstractive system by at least 2 ROUGE points.

Our hybrid *pointer-generator* network facilitates copying words from the source text via *pointing* (Vinyals et al., 2015), which improves accuracy and handling of OOV words, while retaining the ability to *generate* new words. The network, which can be viewed as a balance between extractive and abstractive approaches, is similar to Gu et al.’s (2016) CopyNet and Miao and Blunsom’s (2016) Forced-Attention Sentence Compression,

that were applied to short-text summarization. We propose a novel variant of the *coverage vector* (Tu et al., 2016) from Neural Machine Translation, which we use to track and control coverage of the source document. We show that coverage is remarkably effective for eliminating repetition.

2 Our Models

In this section we describe (1) our baseline sequence-to-sequence model, (2) our pointer-generator model, and (3) our coverage mechanism that can be added to either of the first two models. The code for our models is available online.¹

2.1 Sequence-to-sequence attentional model

Our baseline model is similar to that of Nallapati et al. (2016), and is depicted in Figure 2. The tokens of the article w_i are fed one-by-one into the encoder (a single-layer bidirectional LSTM), producing a sequence of *encoder hidden states* h_i . On each step t , the decoder (a single-layer unidirectional LSTM) receives the word embedding of the previous word (while training, this is the previous word of the reference summary; at test time it is the previous word emitted by the decoder), and has *decoder state* s_t . The *attention distribution* a^t is calculated as in Bahdanau et al. (2015):

$$e_i^t = v^T \tanh(W_h h_i + W_s s_t + b_{\text{attn}}) \quad (1)$$

$$a^t = \text{softmax}(e^t) \quad (2)$$

where v , W_h , W_s and b_{attn} are learnable parameters. The attention distribution can be viewed as

¹www.github.com/abisee/pointer-generator

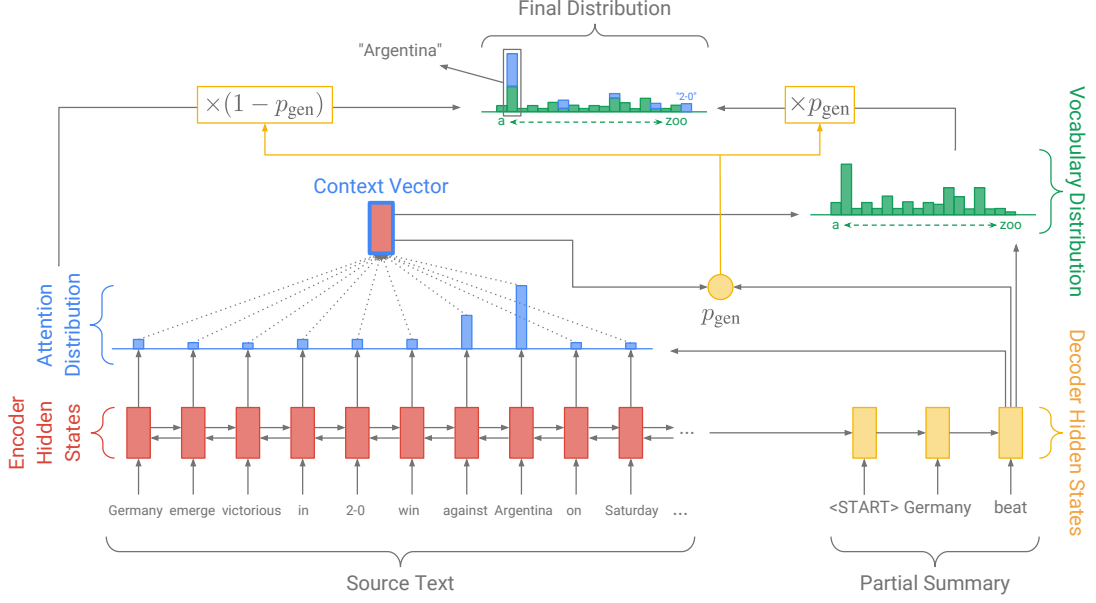


Figure 3: Pointer-generator model. For each decoder timestep a generation probability $p_{\text{gen}} \in [0, 1]$ is calculated, which weights the probability of *generating* words from the vocabulary, versus *copying* words from the source text. The vocabulary distribution and the attention distribution are weighted and summed to obtain the final distribution, from which we make our prediction. Note that out-of-vocabulary article words such as 2-0 are included in the final distribution. Best viewed in color.

a probability distribution over the source words, that tells the decoder where to look to produce the next word. Next, the attention distribution is used to produce a weighted sum of the encoder hidden states, known as the *context vector* h_t^* :

$$h_t^* = \sum_i a_i^t h_i \quad (3)$$

The context vector, which can be seen as a fixed-size representation of what has been read from the source for this step, is concatenated with the decoder state s_t and fed through two linear layers to produce the vocabulary distribution P_{vocab} :

$$P_{\text{vocab}} = \text{softmax}(V'(V[s_t, h_t^*] + b) + b') \quad (4)$$

where V , V' , b and b' are learnable parameters. P_{vocab} is a probability distribution over all words in the vocabulary, and provides us with our final distribution from which to predict words w :

$$P(w) = P_{\text{vocab}}(w) \quad (5)$$

During training, the loss for timestep t is the negative log likelihood of the target word w_t^* for that timestep:

$$\text{loss}_t = -\log P(w_t^*) \quad (6)$$

and the overall loss for the whole sequence is:

$$\text{loss} = \frac{1}{T} \sum_{t=0}^T \text{loss}_t \quad (7)$$

2.2 Pointer-generator network

Our pointer-generator network is a hybrid between our baseline and a pointer network (Vinyals et al., 2015), as it allows both copying words via pointing, and generating words from a fixed vocabulary. In the pointer-generator model (depicted in Figure 3) the attention distribution a^t and context vector h_t^* are calculated as in section 2.1. In addition, the *generation probability* $p_{\text{gen}} \in [0, 1]$ for timestep t is calculated from the context vector h_t^* , the decoder state s_t and the decoder input x_t :

$$p_{\text{gen}} = \sigma(w_{h^*}^T h_t^* + w_s^T s_t + w_x^T x_t + b_{\text{ptr}}) \quad (8)$$

where vectors w_{h^*} , w_s , w_x and scalar b_{ptr} are learnable parameters and σ is the sigmoid function. Next, p_{gen} is used as a soft switch to choose between *generating* a word from the vocabulary by sampling from P_{vocab} , or *copying* a word from the input sequence by sampling from the attention distribution a^t . For each document let the *extended vocabulary* denote the union of the vocabulary, and all words appearing in the source document. We obtain the following probability distribution over the extended vocabulary:

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i: w_i = w} a_i^t \quad (9)$$

Note that if w is an out-of-vocabulary (OOV) word, then $P_{\text{vocab}}(w)$ is zero; similarly if w does